Ministry of Education, Culture and Research of the Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatic

ANDREI ELCISCEV
Group FAF-243

LABORATORY WORK No. 11
THE RACE AND THE SEMAPHORE

Checked by:
Reiţman Patricia

Chişinău, 2026

Contents

1 Introduction **2**

2 Part 1: Observe the Race **2**

3 Part 2: Protect the Counter **2**
3.1 Mutex and One-Permit Semaphore 2
3.2 Semaphore Initially Set to Two 3
3.3 The Cost of Synchronization 3

4 Part 3: Use a Semaphore as a Signal **4**

5 Part 4: Count Free Parking Spaces **5**

6 Conclusion **6**

A Submitted Source Code **6**

1 Introduction

Two POSIX threads each increment a shared counter one million times. The experiments compare an unprotected counter with a mutex and a semaphore, then use semaphores to signal that a message is ready and to limit occupancy in a simulated parking lot.

The programs ran in the Linux VM under `/home/user/os-lab11`. All five programs were compiled with `gcc -Wall -pthread` at the compiler's default optimization level, with no `-O2`. The listings show actual commands and output. The altered initial values were tested in temporary copies of the source files.

```
$ pwd
/home/user/os-lab11
$ uname -sr
Linux 6.14.0-37-generic
$ nproc
3
$ gcc -dumpfullversion
13.3.0
```

Listing 1: Linux VM and compiler

2 Part 1: Observe the Race

```
$ gcc -Wall -pthread -o race race.c
$ ./race
counter = 1675785 (expected 2000000)
$ ./race
counter = 1925845 (expected 2000000)
$ ./race
counter = 1205596 (expected 2000000)
```

Listing 2: Three trials without synchronization

Observe 1. Incrementing the counter consists of loading its value, adding one, and storing the result. These steps do not form an atomic operation.

Observe 2. Starting from 10, both threads can load 10, compute 11, and store 11. Two increments have run, but one store overwrote the other update, leaving 11 instead of 12.

Observe 3. Scheduling and execution overlap vary across trials, so the workers can lose a different number of updates. This is a C data race and has undefined behavior. All three measured trials fell short of 2,000,000.

3 Part 2: Protect the Counter

3.1 Mutex and One-Permit Semaphore

A mutex encloses the read, addition, and store in one critical section. The semaphore version also admits one thread at a time when initialized to one. `sem_wait` acquires the

permit and `sem_post` returns it.

```
$ gcc -Wall -pthread -o mutex mutex.c
$ ./mutex
counter = 2000000 (expected 2000000)
$ ./mutex
counter = 2000000 (expected 2000000)
$ ./mutex
counter = 2000000 (expected 2000000)
$ gcc -Wall -pthread -o sem sem.c
$ ./sem
counter = 2000000 (expected 2000000)
$ ./sem
counter = 2000000 (expected 2000000)
$ ./sem
counter = 2000000 (expected 2000000)
```

Listing 3: Three correctness trials per lock

Observe 1. Both protected versions reported 2,000,000 in all three requested trials and in the three additional timed trials.

3.2 Semaphore Initially Set to Two

```
$ sed 's/sem_init(&s, 0, 1)/sem_init(\&s, 0, 2)/' sem.c > sem2.c
$ grep -n sem_init sem2.c
23:    sem_init(&s, 0, 2);         /* 0 = shared between threads, 1 = start value */
$ gcc -Wall -pthread -o sem2 sem2.c
$ ./sem2
counter = 1999341 (expected 2000000)
$ ./sem2
counter = 1947310 (expected 2000000)
$ ./sem2
counter = 1994467 (expected 2000000)
```

Listing 4: Two permits admit both workers

Observe 2. Both workers can acquire a permit and enter the critical section together. The read–modify–write race returns, and all three trials lose increments. The submitted `sem.c` uses one permit.

3.3 The Cost of Synchronization

```
$ time ./race
counter = 1832162 (expected 2000000)
real 0.015 s | user 0.015 s | sys 0.002 s
$ time ./mutex
counter = 2000000 (expected 2000000)
real 0.046 s | user 0.060 s | sys 0.017 s
$ time ./sem
counter = 2000000 (expected 2000000)
real 0.074 s | user 0.087 s | sys 0.044 s
```

```
$ time ./race
counter = 1269920 (expected 2000000)
real 0.010 s | user 0.014 s | sys 0.002 s
$ time ./mutex
counter = 2000000 (expected 2000000)
real 0.058 s | user 0.055 s | sys 0.016 s
$ time ./sem
counter = 2000000 (expected 2000000)
real 0.072 s | user 0.072 s | sys 0.018 s
$ time ./race
counter = 1269203 (expected 2000000)
real 0.017 s | user 0.016 s | sys 0.006 s
$ time ./mutex
counter = 2000000 (expected 2000000)
real 0.069 s | user 0.068 s | sys 0.015 s
$ time ./sem
counter = 2000000 (expected 2000000)
real 0.072 s | user 0.081 s | sys 0.050 s
```

Listing 5: Three wall-time measurements per version

Observe 3. Acquiring and releasing a lock on every increment adds synchronization operations. When workers contend, the shared lock state also causes cache-coherence traffic and can require waits and wakeups. The median wall times in this VM were 0.015 s for the race, 0.058 s for the mutex, and 0.072 s for the semaphore.

4 Part 3: Use a Semaphore as a Signal

The reader waits on `ready`. With an initial value of zero, the writer stores the message and posts before the reader can acquire a permit and read it.

```
$ gcc -Wall -pthread -o order order.c
$ ./order
B: waiting for the message...
A: message written
B: I read "hello"
$ sed 's/sem_init(&ready, 0, 0)/sem_init(\&ready, 0, 1)/' order.c > order1.c
$ grep -n sem_init order1.c
30:    sem_init(&ready, 0, 1);
$ gcc -Wall -pthread -o order1 order1.c
$ ./order1
B: waiting for the message...
B: I read ""
A: message written
```

Listing 6: Signal experiment with initial values zero and one

Observe 1. With an initial value of one, the reader consumed the initial permit and printed an empty string before the writer ran its store. The global array starts with zero bytes. This variant removes the synchronization ordering between the writer's store and the reader's load, so the empty string describes this run.

Observe 2. A semaphore initialized to zero makes a waiter wait for a post to supply a permit. The submitted version starts at zero and the reader printed `hello` after the writer's message.

5 Part 4: Count Free Parking Spaces

Each of the six car threads acquires a permit before entering and returns it after leaving. The mutex protects the occupancy count and the associated print.

```
$ gcc -Wall -pthread -o parking parking.c
$ ./parking
car 1 enters  (inside: 1)
car 3 enters  (inside: 2)
car 2 enters  (inside: 3)
car 2 leaves  (inside: 2)
car 3 leaves  (inside: 1)
car 5 enters  (inside: 2)
car 4 enters  (inside: 3)
car 1 leaves  (inside: 2)
car 6 enters  (inside: 3)
car 4 leaves  (inside: 2)
car 6 leaves  (inside: 1)
car 5 leaves  (inside: 0)
```

Listing 7: Parking with three places

```
$ sed 's/^#define PLACES 3$/#define PLACES 1/' parking.c > parking1.c
$ grep -n #define PLACES parking1.c
9:#define PLACES 1
$ gcc -Wall -pthread -o parking1 parking1.c
$ ./parking1
car 1 enters  (inside: 1)
car 1 leaves  (inside: 0)
car 2 enters  (inside: 1)
car 2 leaves  (inside: 0)
car 4 enters  (inside: 1)
car 4 leaves  (inside: 0)
car 5 enters  (inside: 1)
car 5 leaves  (inside: 0)
car 6 enters  (inside: 1)
car 6 leaves  (inside: 0)
car 3 enters  (inside: 1)
car 3 leaves  (inside: 0)
```

Listing 8: Parking with one place

Observe 1. The maximum recorded occupancy was three. `sem_init(&places, 0, PLACES)` provides three permits, and `sem_wait(&places)` gates entry. A car releases its permit with `sem_post(&places)` after leaving.

Observe 2. With one place, the semaphore functions as a binary semaphore for the slot. The trace shows one car at a time. All six cars entered and left once in each experiment, and both runs ended with zero cars inside. The submitted version uses three places.

6 Conclusion

The mutex and the semaphore with one permit prevented lost increments. Starting the message semaphore at zero enforced the ordering between writer and reader. Starting the parking semaphore at three bounded concurrent occupancy. Each use depends on when a thread acquires or returns a permit and how many permits are available initially.

A Submitted Source Code

```
/* race.c - two threads add to the same counter, with no protection.
 * Build:  gcc -Wall -pthread -o race race.c      (no -O2: keep counter++ in memory)
 * Run:    ./race   (several times)
 */
#include <stdio.h>
#include <pthread.h>

#define N 1000000

long counter = 0;                 /* shared by both threads */

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        counter++;                /* read, add 1, write back: three steps */
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

Listing 9: race.c

```
/* mutex.c - the same counter, now protected by a mutex.
 * Build:  gcc -Wall -pthread -o mutex mutex.c
 */
#include <stdio.h>
#include <pthread.h>

#define N 1000000

long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);      /* only one thread past this line */
        counter++;                      /* the critical section */
        pthread_mutex_unlock(&lock);    /* let the other one in */
```

```
    }
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

Listing 10: mutex.c

```
/* sem.c - the same counter, protected by a semaphore that starts at 1.
 * Build:  gcc -Wall -pthread -o sem sem.c
 */
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define N 1000000

long counter = 0;
sem_t s;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        sem_wait(&s);           /* 1 -> 0: I go in; at 0: wait */
        counter++;
        sem_post(&s);           /* 0 -> 1: the next one may go in */
    }
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 1);         /* 0 = shared between threads, 1 = start value */
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    sem_destroy(&s);
    return 0;
}
```

Listing 11: sem.c

```
/* order.c - thread B must wait until thread A has written the message.
 * A semaphore that starts at 0 is a signal: sem_wait blocks until someone posts.
 * Build:  gcc -Wall -pthread -o order order.c
 */
#include <stdio.h>
#include <string.h>
```

```c
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

char message[32];
sem_t ready;                    /* starts at 0: nothing is ready yet */

void *writer(void *arg) {
    sleep(1);                   /* pretend the work takes a while */
    strcpy(message, "hello");
    printf("A: message written\n");
    sem_post(&ready);           /* 0 -> 1: wake up B */
    return NULL;
}

void *reader(void *arg) {
    printf("B: waiting for the message...\n");
    sem_wait(&ready);           /* blocks while the value is 0 */
    printf("B: I read \"%s\"\n", message);
    return NULL;
}

int main(void) {
    sem_init(&ready, 0, 0);
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);   /* B starts first, on purpose */
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    sem_destroy(&ready);
    return 0;
}
```

Listing 12: order.c

```c
/* parking.c - 6 cars, 3 places. A semaphore that starts at 3 counts the free places.
 * Build:  gcc -Wall -pthread -o parking parking.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define PLACES 3
#define CARS   6

sem_t places;                                   /* free places */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int inside = 0;                                 /* only for printing */

void *car(void *arg) {
    long id = (long)arg;
    sem_wait(&places);                          /* take a place, or wait */
    pthread_mutex_lock(&lock);
    inside++;
    printf("car %ld enters  (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);
```

```
    sleep(1);                                   /* parked */

    pthread_mutex_lock(&lock);
    inside--;
    printf("car %ld leaves  (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);
    sem_post(&places);                          /* give the place back */
    return NULL;
}

int main(void) {
    sem_init(&places, 0, PLACES);
    pthread_t t[CARS];
    for (long i = 0; i < CARS; i++)
        pthread_create(&t[i], NULL, car, (void *)(i + 1));
    for (int i = 0; i < CARS; i++)
        pthread_join(t[i], NULL);
    sem_destroy(&places);
    return 0;
}
```

Listing 13: parking.c